Snake Game Development Log

User Write a basic Python script using the Pygame library. Initialize a window of 600x400 pixels. Create a main game loop that handles the QUIT event so the window closes properly. Fill the background with black. Limit the frame rate to 10 FPS.

Gemini Building a basic shell in Pygame is a great way to understand how game engines handle real-time updates. Here is a clean, well-commented script that meets all your requirements.

Basic Pygame Boilerplate

```
import pygame
import sys

# 1. Initialize Pygame
pygame.init()

# 2. Set up the display (Width, Height)
screen = pygame.display.set_mode((600, 400))
pygame.display.set_caption("My Basic Game Loop")

# 3. Create a clock object to control the frame rate
clock = pygame.time.get_ticks()
# We use pygame.time.Clock() to manage the frame rate effectively
timer = pygame.time.Clock()

# Main Game Loop
running = True
while running:
    # 4. Event Handling
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # 5. Drawing / Rendering
    # Fill the background with black (RGB: 0, 0, 0)
    screen.fill((0, 0, 0))

    # Update the full display Surface to the screen
    pygame.display.flip()

    # 6. Cap the frame rate at 10 FPS
    timer.tick(10)

# 7. Clean shutdown
pygame.quit()
sys.exit()
```

User *[User provides code snippet for "Fancy Moving Snake"]* so far i have this but now: Write a function to spawn food. It should return a random (x, y) coordinate. IMPORTANT: The coordinate must be a multiple of 20 (my block size) and stay within the 600x400 window. Draw this food as a red rectangle.

Gemini You've built a solid foundation! Using a list of tuples for the snake body is the standard way to handle growth later on.

To add food, we need to ensure it snaps to the same grid as your snake segments. Since your `BLOCK_SIZE` is 20, the x-coordinates must be multiples of 20 between `0` and `WIDTH - 20`, and y-coordinates between `0` and `HEIGHT - 20`.

Updated Script with Food Logic

```
import pygame
import sys
import random # 1. Import random

# ... (Constants Omitted for brevity) ...

# 2. Function to spawn food on the grid
def spawn_food():
    # We subtract BLOCK_SIZE from the max range so it doesn't spawn off-screen
    x = random.randrange(0, WIDTH, BLOCK_SIZE)
    y = random.randrange(0, HEIGHT, BLOCK_SIZE)
    return (x, y)

def draw_food(food_pos):
    rect = pygame.Rect(food_pos[0], food_pos[1], BLOCK_SIZE, BLOCK_SIZE)
    pygame.draw.rect(screen, RED, rect, border_radius=8) # Slightly rounded for c

def main():
    global x_change, y_change

    # 3. Initialize food position
    food_pos = spawn_food()

    running = True
    while running:
        # ... (Event Handling Omitted) ...

        # --- Update Snake Position ---
        new_head = (snake_body[0][0] + x_change, snake_body[0][1] + y_change)
        snake_body.insert(0, new_head)

        # 4. Check for Food Collision
        if snake_body[0] == food_pos:
            # Snake ate the food! Don't pop the tail, and spawn new food.
            food_pos = spawn_food()
        else:
```

```
        # Normal movement: remove the tail
        snake_body.pop()

    # ... (Drawing Section) ...
```

User I want you to add that prompt into my previous code

Gemini Here is your updated code with the `spawn_food` function and the logic to grow the snake integrated. I've placed the `spawn_food` function at the top level and updated the `main()` loop to handle the collision logic.

[Full Code Provided - See final version for latest iteration]

User Check for collision between the Snake's head (index 0) and the food position. If they match:

1. Print 'Yummy!' to the console.
2. Do NOT remove the tail segment this frame (this makes the snake grow).
3. Generate a new food position.

Gemini I've integrated the collision logic exactly as you requested. By wrapping the `.pop()` method in an `else` statement, the snake naturally grows because the new head was already inserted.

```
    # 3. Collision Logic (The logic you requested)
    if snake_body[0] == food_pos:
        print("Yummy!")
        food_pos = spawn_food()
        # Note: We do NOT pop the tail here, so the snake grows
    else:
        # Remove the tail segment to maintain length during normal movement
        snake_body.pop()
```

User Add boundary checks. If the snake's head x coordinate is < 0 or >= 600, or y < 0 or >= 400, stop the game loop or print 'Game Over'.

Gemini To keep the game clean and functional, I've added the boundary checks right after the snake updates its position. I've also added a "Self-Collision" check.

```
    # 2. BOUNDARY CHECKS
    if (new_head[0] < 0 or new_head[0] >= WIDTH or
        new_head[1] < 0 or new_head[1] >= HEIGHT):
        print("Game Over: Hit the Wall!")
```

```
            running = False
            continue # Skip the rest of the loop to exit
```

User Check if the snake hits itself. If the new head position exists anywhere in the current snake_body list (excluding the head itself), trigger Game Over.

Gemini I've integrated the self-collision check. This is done by checking if the `new_head` coordinate already exists within the `snake_body` list before we actually insert it.

```
        # 3. SELF-COLLISION CHECK
        if new_head in snake_body:
            print("GAME OVER: You bit your own tail!")
            running = False
```

User when I die don't turn off the window and pop up "Game Over!" on the screen. Note: Don't turn of the window until I do by myself

Gemini To keep the window open after a collision, we need to change how the game handles the "Game Over" state. Instead of breaking the loop, we will introduce a `game_over` state where the snake stops moving.

```
def show_game_over():
    """Renders the Game Over text in the center of the screen."""
    text_surface = font.render("GAME OVER!", True, WHITE)
    # ... logic to blit text ...
```

User I want to display the score (Length of snake - 3) in the top-left corner. Use pygame.font.SysFont. Write a function show_score() and call it after drawing the background. I want to add historic score board as a "highest score" which will be highest in the past and whenever beats that, it should change and current score board as well

Gemini To handle a persistent high score, we need to save the data to a file (like a simple text file). I have added the `show_score()` function and logic to manage a `high_score.txt` file.

```
HS_FILE = "high_score.txt"

def load_high_score():
    # ... logic to read file ...

def save_high_score(score):
```

```
        # ... logic to write file ...
```

User I want to add start page which has start button and an end page after 3 seconds of game over add restart button or end the gamke button

Gemini To implement a **Start Page** and an **End Page**, we need to introduce a "State Machine." This allows the game to switch between different modes (e.g., `START`, `PLAYING`, `GAME_OVER`).

[Code updated with State Machine logic 'START', 'PLAYING', 'GAMEOVER']

User *[User pastes previous code]* I like everything you gave me before but based on this code add the code because I lost the snake image and boundaries

Gemini It looks like we are building the ultimate version of your Snake game! I have integrated the **Start Page**, the **Score/High Score system**, the **Boundary/Self-collision checks**, and the **End Page** into your current code structure.

User based on previous code I want to add automatic difficulty after eating each apple movement should increase a little bit but at the begining it should be slow

Gemini I've updated the script to start at a relaxed pace (7 FPS) and increase by a small amount every time the snake eats. I also capped the speed so it doesn't eventually become physically impossible to play.

```
# Difficulty Settings
START_SPEED = 7   # Slow start
SPEED_INC   = 0.5 # Increase per apple
MAX_SPEED   = 25  # Speed cap
```

User based on previous code I want to change something. after i die i get game over but for the end page it takes 3 seconds I want to change this it should not take any second

Gemini Understood! We can remove the `time.sleep()` or the `time.time()` comparison entirely. By removing that delay, the **Restart** and **Quit** buttons will appear the exact millisecond the snake's head hits a boundary.

User based on previous code I want to have three level in this game, Easy, Medium, and hard buttons in the first page.

Gemini To implement levels, we need to adjust the `START_SPEED` and `SPEED_INC` (speed increment) based on which button the user clicks on the start page.

User Easy: In easy part I want to start from speed 7 and increase by 1. Medium: I want to have border on random places, and my fruit can't be on border. speed should start from 8. Hard: In Hard part I want to have puzzle border it should be the same puzzle every time and fruit shouldn't be on border puzzle. speed should start from 9. Note: don't change previous code. but I want to change one thing that if I check for Arrow Key presses (UP, DOWN, LEFT, RIGHT) and update x_change / y_change. Prevent the snake from reversing into itself (e.g., can't go Left if going Right).

Gemini I have updated the logic to include:

1. **Dynamic Difficulty**: Speed and increments set per level.
2. **Obstacle Logic**: `Medium` generates random blocks; `Hard` uses a fixed "Puzzle" layout.
3. **Smart Food Spawning**: A loop that ensures food never lands on a snake or a wall.
4. **Input Validation**: A strict check to prevent the snake from 180-degree self-reversal.

User everything is fine but I want to convert the medium part into hard part

Gemini I have adjusted the levels so that **Hard** now features the random obstacle generation, while **Medium** is a cleaner, faster version of the standard game.

User now for the medium have 3 straight borders everything else is fine

Gemini I have updated the **Medium** level to include three straight borders. I've set them up as three horizontal bars that act as obstacles in the middle of the screen.

User there is a problem with medium I can't open it fix this and add: I could be able to do select the mode with keyboard buttons e.g for easy E, for medium M, and for hard H. and for restart R and for quit Q

Gemini I have streamlined the logic to ensure that when you press a button (or a key), the game fully resets all variables before switching the state to `PLAYING`. I have also added the **Keyboard Shortcuts** you requested.

User I still can't open the medium mode. I should be able to play like easy and hard one. For the hard one: have 10 random blocks on random places each time but should be there where snake start

Gemini I have refactored the code so that **Level Data (Speed, Obstacles, and Food) is generated exactly once** the moment you click or press the key. I also reduced the Hard mode to **10 random blocks** as requested.

User Medium three staight lines at same place everytime but lines should not be there when I start the mode

Gemini I understand exactly what you're looking for. The issue was that the snake starts at `(100, 100)`, and one of the lines was also spawning at `y = 100`, causing an instant crash. I have shifted the three lines so they are away from the snake's starting path.

User Modify: Easy: I want to increase speed by 0.5 only for easy hard: I want to I want to have 15 blocks on random places each time but should be there where snake start

Gemini To make these changes, I have updated the **Easy** speed increment to exactly `0.5` and modified the **Hard** mode to generate **15 random blocks** while ensuring they never spawn on top